



LangChain

Created by : Mohammed Ammar Karimi

Agenda

- Introduction
- Requirements
- Types of Prompting Techniques
- RAG
- Demo





Introduction

What is Langchain

- LangChain is an open-source framework that allows developers to build applications using large language models (LLMs).
- It provides tools and APIs to build applications like chatbots and virtual agents.
- LangChain helps developers improve the accuracy and relevancy of the information generated by LLM using different prompting techniques.
- LangChain offers libraries in Python, JavaScript, and TypeScript.



Requirements

How to install

What libraries needed to create a basic langchain application.

Below are **System setup**:

- Python
 - <https://www.python.org/>
- Any editor (VS code or jupyter)
 - Python extension in VS code

Once system is setup open a project folder where you want to create the project

- Install langchain and langchain-community
 - pip install langchain
 - pip install langchain-community
- Install openai
 - pip install openai
- While running code if 'module not found' error arise then use 'pip' command to install that module.

Types of Prompting



ZeroShot and FewShot

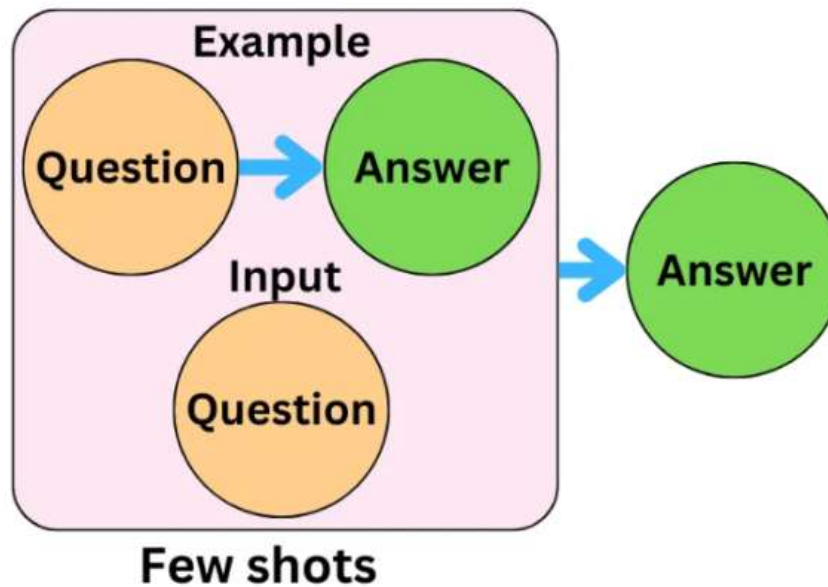
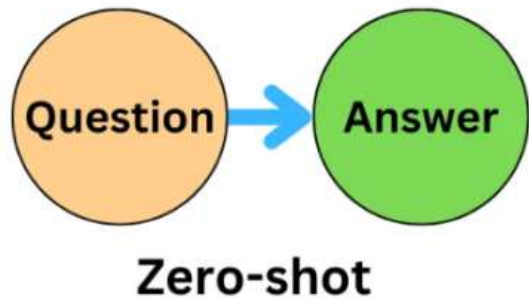
ZeroShot

- In zero-shot prompting technique, we do not guide the LLM with examples.
- It generates responses purely based on its pre-trained knowledge and understanding of the prompt.

FewShot

- In few-shot prompting, we provide the LLM with a few examples to guide its response.
- These examples help the model understand the desired format, tone, or reasoning, leading to more accurate and context-aware outputs.

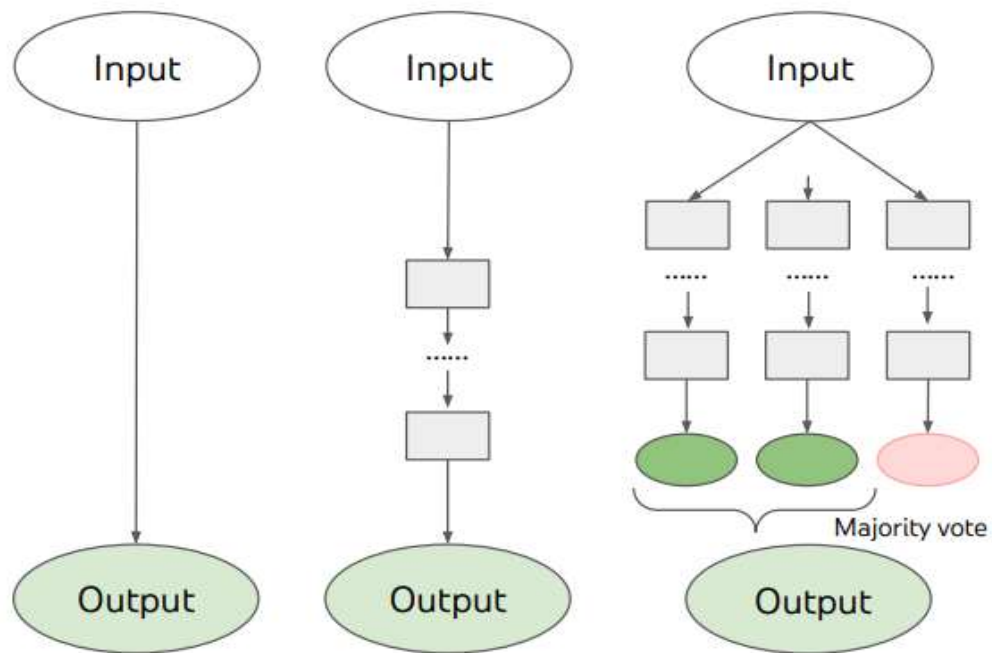
ZeroShot and FewShot



TOT(Tree of Thoughts)

- ToT is an upgraded version of CoT.
- In CoT (Chain of Thoughts) approach, LLMs tend to progress linearly in their thinking towards problem solving, and if an error occurs along the way, they tend to proceed along that erroneous criterion.
- In contrast, in the ToT (Tree of Thoughts) approach, LLMs evaluate themselves at each stage of thought and stop inefficient approaches early, switching to alternative methods
- Research Paper: <https://arxiv.org/pdf/2305.10601>
- COT is a step-by-step approach helps the model handle complex problems by breaking them down into manageable parts.
- ToT extends the CoT approach by allowing the model to explore multiple reasoning paths simultaneously, forming a tree-like structure.
- At each decision point, the model can consider various possible next steps, evaluate them, and choose the most promising path.
- This method enables the model to backtrack and explore alternative solutions, enhancing its problem-solving abilities.

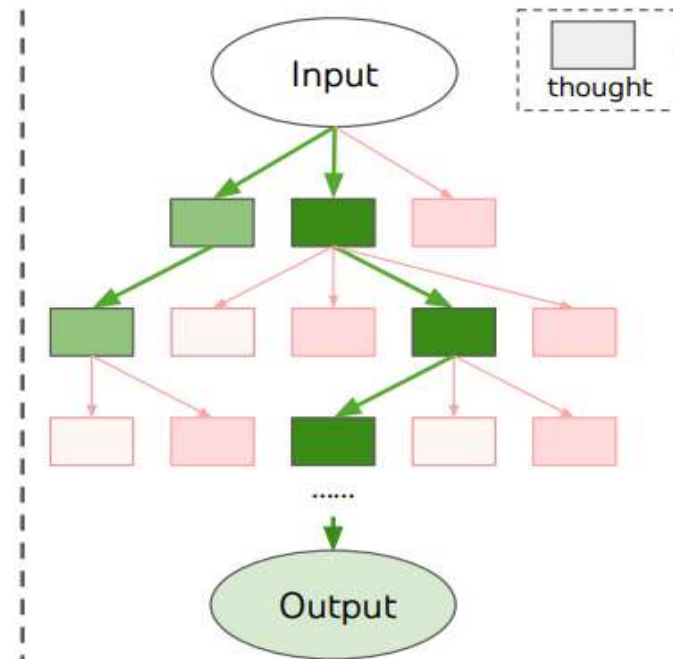
TOT(Tree of Thoughts)



(a) Input-Output Prompting (IO)

(b) Chain of Thought Prompting (CoT)

(c) Self Consistency with CoT (CoT-SC)



(d) Tree of Thoughts (ToT)

TOT(Tree of Thoughts)

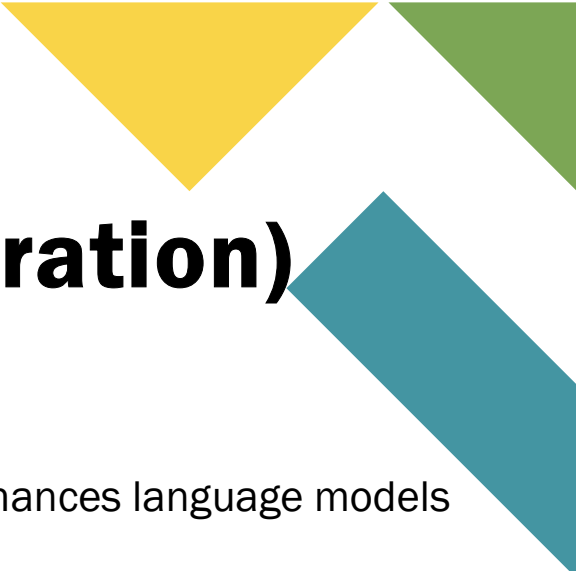
- In Tree-of-Thought (ToT) prompting, the model generates multiple potential solutions, ranking them based on their effectiveness
- This approach not only helps in identifying the most accurate answer but also provides alternative solutions if the initial one does not work
- By exploring different reasoning paths, ToT increases the chances of finding the best possible outcome while allowing flexibility in problem-solving.

```
"ranked_solutions": {
  "Ranking_1": {
    "Justification": "Using In-Situ Resource Utilization is the most promising solution for Mars colonization.",
    "In_Situ_Resource_Utilization_ISRU": {
      "Pros": "This solution could provide the necessary resources for Mars colonization.",
      "Cons": "ISRU is technically challenging and would require significant initial effort.",
      "Initial_Effort": "Medium. This would require the development of advanced technologies.",
      "Implementation_Difficulty": "Medium. ISRU is a complex task that requires careful planning.",
      "Potential_Challenges": "Technical difficulties, high costs.",
      "Expected_Outcomes": "If successful, ISRU could provide a steady supply of resources.",
      "Probability_of_Success": "High. ISRU is already being tested by NASA's Perseverance rover.",
      "Confidence_Level": "70%"
    }
  },
  "Ranking_2": {
    "Justification": "Building a self-sustaining colony is a promising solution for Mars colonization.",
    "Building_a_Self_Sustaining_Colony": {
      "Pros": "This solution could make the Mars colony self-sufficient and reduce the need for resupply.",
      "Cons": "Building a self-sustaining colony is a complex task that requires a large initial effort.",
      "Initial_Effort": "High. This would require the development of advanced technologies.",
      "Implementation_Difficulty": "High. Building a self-sustaining colony is a complex task.",
      "Potential_Challenges": "Technical difficulties, high costs.",
      "Expected_Outcomes": "If successful, a self-sustaining colony could provide a steady supply of resources.",
      "Probability_of_Success": "Medium. While there are significant challenges, it is a promising solution.",
      "Confidence_Level": "60%"
    }
  },
  "Ranking_3": {
    "Justification": "While asteroid mining has the potential to provide resources, it is a high-risk solution.",
    "Terraforming_Mars": {
      "Pros": "This solution could make Mars more habitable for humans and provide a long-term solution.",
      "Cons": "Terraforming is a long-term process that could take centuries and require a large initial effort.",
      "Initial_Effort": "Extremely High. Terraforming would require a massive investment of resources.",
      "Implementation_Difficulty": "Extremely High. Terraforming is a complex task that requires advanced technologies.",
      "Potential_Challenges": "Technical difficulties, high costs, time.",
      "Expected_Outcomes": "If successful, terraforming could make Mars a permanent home for humans.",
      "Probability_of_Success": "Low. Terraforming is a theoretical concept that has not been tested.",
      "Confidence_Level": "20%"
    }
  }
}
```

RAG



RAG (Retrieval Augmented Generation)



- Retrieval Augmented Generation (RAG) is a powerful technique that enhances language models by combining them with external knowledge bases.
- The concept of RAG is relatively straightforward. It involves two main components: A document retriever and a large language model (LLM).
- The **document retriever** is responsible for finding relevant information from a large corpus of documents based on the input question using **semantic search**(Understanding context using BERT like model) or using **similarity search**(Vector based approach like cosine similarity).
- Large Language Models (LLMs) are highly capable but encounter several issues like creating inaccurate or irrelevant content (hallucinations), using outdated information, and operating in ways that are not transparent (blackbox reasoning).
- Retrieval-Augmented Generation (RAG) is a technique to solve these problems by augmenting LLM knowledge with additional domain specific data.

RAG Pipeline



The Basic Retrieval-Augmented Generation (RAG) Pipeline operates through two main phases: **Data Indexing** and **Retrieval and Generation**

1. Data Indexing

1. **Data Loading:** This involves importing all the documents or information to be utilized.
2. **Data Splitting:** Large documents are divided into smaller pieces, for instance, sections of no more than 500 characters each.
3. **Data Embedding:** The data is converted into vector form using an embedding model, making it understandable for computers.
4. **Data Storing:** These vector embeddings are saved in a vector database, allowing them to be easily searched.

RAG Pipeline



2. Retrieval and Generation Process:

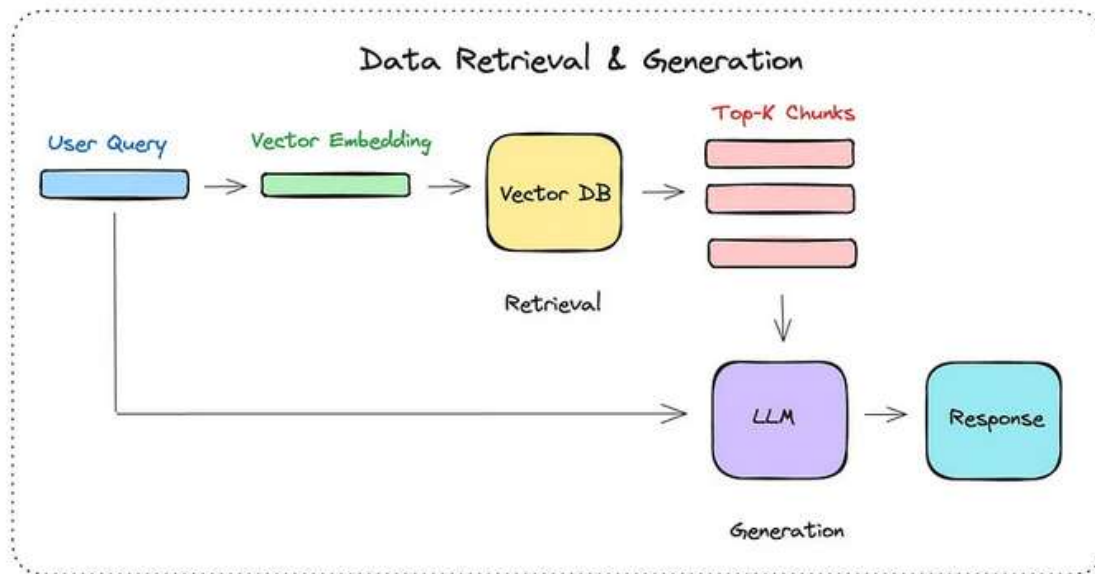
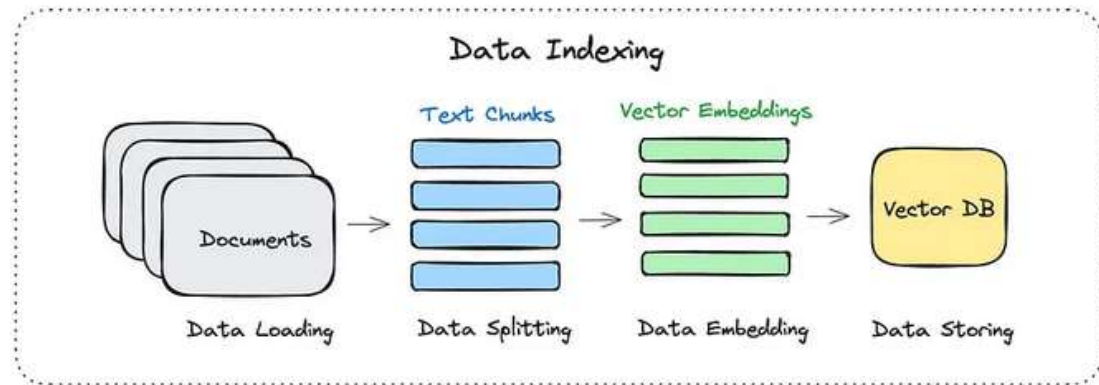
1. Retrieval: When a user asks a question:

- The user's input is first transformed into a vector (query vector) using the same embedding model from the Data Indexing phase.
- This query vector is then matched against all vectors in the vector database to find the most similar ones.

2. Generation Process:

- The LLM model takes the user's question and the relevant information retrieved from the vector database to create a response.
- This process combines the question with the identified data to generate an answer.

RAG Pipeline





Demo

Thank you

